Natural Language Processing

Prof. Ahmed Guessoum
The National Higher School of AI

Chapter 2

Regular Expressions, Tokenization, Edit Distance

Outline

- Regular Expressions
- Words
- Corpora
- Simple Unix Tools for Word Tokenization
- Word and Subword Tokenization
- Word Normalization, Lemmatization and Stemming
- Sentence Segmentation
- Minimum Edit Distance

Basic Text Processing: Regular Expressions

- Regular expressions are used everywhere:
  - Part of every text processing task
    - Not a general NLP solution (for that we use large NLP systems we will see in later lectures)
    - But very useful as part of those systems
      - E.g., searching in text, web scraping, data formatting, text pre-processing.
  - Necessary for data analysis of text data:
    - E.g., pattern-matching
  - A widely used tool in industry and academia

Regular Expressions (RegEx)

- A formal language for specifying text strings
- How can we search for mentions of these cute animals in text?
  - woodchuck
  - woodchucks
  - Woodchuck
  - Woodchucks
  - Groundhog
  - groundhogs



Marmotte in French.

RegEx : Disjunctions

- Letters inside square brackets []

Pattern	Matches
[wW]oodchuck	Woodchuck, woodchuck
[1234567890]	Any one digit

- Ranges using the dash `[A-Z]`

Pattern	Matches	
`[A-Z]`	An upper case letter	`My beans were impatient`
`[a-z]`	A lower case letter	`My beans were impatient`
`[0-9]`	A single digit	`Chapter 1: Down the Rabbit Hole`

RegEx: Negation in Disjunction

- Caret as first character in [] negates the list
  - Note: Caret means negation only when it's first in []
  - Special characters (., *, +, ?) lose their special meaning inside []

Pattern	Matches	Examples
[^A-Z]	Not an upper case letter	Oyfn pripetchik
[^Ss]	Neither 'S' nor 's'	So I have no exquisite reason"
[^.]	Not a period	Our resident Djinn
[e^]	Either e or ^	Look up ^ now

RegEx : Convenient aliases

Pattern	Expansion	Matches	Examples
\d	[0-9]	Any digit	Fahreneit 451
\D	[^0-9]	Any non-digit	Blue Moon
\w	[a-zA-Z0-9_]	Any alphanumeric or _	Daiyu
\W	[^\w]	Not alphanumeric or _	Look!
\s	[\r\t\n\f]	Whitespace (space, carriage return, tab, newline, form feed)	Look␣up
\S	[^\s]	Not whitespace	Look up

RegEx : More Disjunction

- Groundhog is another name for woodchuck!
- The pipe symbol | for disjunction

Pattern	Matches
groundhog|woodchuck	woodchuck
yours|mine	yours
a|b|c	= [abc]
[gG]roundhog|[Ww]oodchuck	Woodchuck



Wildcards, optionality, repetition: . ? * +

Pattern	Matches	Examples
beg.n	Any char	begin begun beg3n beg n
woodchucks?	Optional s	woodchuck woodchucks
to*	0 or more of previous char	t to too tooo
to+	1 or more of previous char	to too tooo toooo



Stephen C Kleene

RegEx : Anchors ^ $

Pattern	Matches
^[A-Z]	Palo Alto
^[^A-Za-z]	1 "Hello"
\.$	The end.
.$	The end? The end!

The caret matches the start of the line, and the $ matches the end of the line.

A note about Python RegEx

- In **Python**, backslashes (\) are used both:
  - by **Python strings** (to introduce escape sequences like \n or \t), **and**
  - by **regular expressions** (to introduce regex escapes like \d, \w, \s, etc.).
- Writing a **normal Python string**, is ≡ actually writing something that gets *interpreted twice*:
  - Once by Python (to create the string in memory),
  - Then by the regex engine (to interpret the pattern).

A note about Python RegEx

- Writing the string "\\d+", Python sees:
  - \\ → one literal backslash (\)
  - d+ → the letter d followed by +

  So, the resulting string *in memory* is actually: \d+
- "\n" in Python means the "newline" character, not a "backslash" followed by an "n" ➔ Need "\\n" for the two characters.

A note about Python RegEx

- To avoid the double escaping confusion, **Python provides "raw strings"**, written with an r prefix: r"\d+"
- This tells Python:

  "Do not interpret backslashes as escape sequences; pass them as-is."
- So now the regex engine receives exactly what you typed.
  - r"[tT]he"
  - r"\d+" matches one or more digits
    - instead of "\\d+"

The iterative process of writing regex's

- Find me all instances of the word “the” in a text.

the	Misses capitalized examples
[tT]he	Incorrectly returns “other” or “Theology”
\W[tT]he\W	

False positives and false negatives

The process we just went through was based on fixing two kinds of errors:

1. Not matching things that we should have matched (The)

   False negatives

2. Matching strings that we should not have matched (there, then, other)

   False positives

Characterizing work on NLP

- In NLP we are always dealing with these kinds of errors.
- Reducing the error rate for an application often involves two antagonistic efforts:
  - Increasing coverage (or *recall*) (minimizing false negatives).
  - Increasing accuracy (or *precision*) (minimizing false positives)

Regular expressions play a surprisingly large role

- Widely used in both academia and industry

1. Part of most text processing tasks, even for big neural language model pipelines
    - including text formatting and pre-processing
2. Very useful for data analysis of any text data

Basic Text Processing:
More Regular Expressions – Substitutions and ELIZA

Basic Text Processing: Substitutions

- Substitution in UNIX commands:
  - s/regexp1/pattern/

  change a string matched by a regex to substitute another string.

  e.g.: s/colour/color/
- In Python:

```
import re
text = "The colour of the sky is beautiful."
result = re.sub(r'colour', 'color', text)
print(result)
 #Outputs: The color of the sky is beautiful.
```

Capturing Groups

- Say we want to put angles around all numbers:

  the 35 boxes → *the <35> boxes*

- Use parentheses () to "capture" a pattern into a numbered register (1, 2, 3…)
- Use \1 to refer to the contents of the register

s/([0-9]+)/<\1>/

- In Python:

```
result2 = re.sub(r'(\d+)',r'<\1>',text2)
```

Capturing groups: multiple registers

- /the (.*)er they (.*), the \1er we \2/

  Matches

- *the faster they ran, the faster we ran*

  But not

- *the faster they ran, the faster we ate*

But suppose we don't want to capture?

- Parentheses have a double function: grouping terms, and capturing
- Non-capturing groups: add a ?: after parentheses:

  /(?:some|a few) (people|cats) like some \1/
- matches
  - some cats like some cats
- but not
  - some cats like some people

Lookahead assertions

- (?= pattern) is true if pattern matches, but is **zero-width; doesn't advance character pointer**
- (?! pattern) true if a pattern does not match
- How to match, at the beginning of a line, any single word that doesn’t start with “Volcano”:

  /^(?!Volcano)[A-Za-z]+/

- In Python:

```
# Pattern to match "First" at the beginning of a
line
pattern = r'^First'
# Using re.MULTILINE to allow ^ to match the start
# of each line (e.g. text string on several lines)
matches = re.findall(pattern, text, re.MULTILINE)
```

Example of Python Reg. Exp. Script

- Sample Python script to use regular expressions:

Simple Application: ELIZA

- Early NLP system that imitated a Rogerian psychotherapist. Developed by Joseph Weizenbaum, 1966.

- Uses pattern matching to match, e.g.,:
  - "I need X"

and translates them into, e.g.

  - "What would it mean to you if you got X?

Simple Application: ELIZA

Men are all alike.

IN WHAT WAY

They're always bugging us about something or other.

CAN YOU THINK OF A SPECIFIC EXAMPLE

Well, my friend made me come here.

YOUR FRIEND MADE YOU COME HERE

He says I'm depressed much of the time.

I AM SORRY TO HEAR YOU ARE DEPRESSED

How ELIZA works

- s/.* I'M (depressed|sad) .*/I AM SORRY TO HEAR YOU ARE \1/
- s/.* I AM (depressed|sad) .*/WHY DO YOU THINK YOU ARE \1/
- s/.* all .*/IN WHAT WAY?/
- s/.* always .*/CAN YOU THINK OF A SPECIFIC EXAMPLE?/

Words and Corpora

How many words in a sentence?

- "I do uh main- mainly business data processing"
  - Fragments, filled pauses
- "Khaled's cat in the hat is different from other cats!"
  - **Lemma**: same stem, part of speech, rough word sense
    - cat and cats = same lemma
  - **Word form**: the full inflected surface form
    - cat and cats = different word forms

How many words in a sentence?

they lay back on the Hassi Dalla3ah grass and looked at the stars and their

- **Type**: an element of the vocabulary.
- **Token**: an instance of that type in running text.
- How many?
  - 15 tokens (or 14)
  - 13 types (or 12)

How many words in a corpus?

N = number of tokens

V = vocabulary = set of types, **|*V*|** is size of vocabulary

Heaps Law = Herdan's Law = $|V| = kN^{\beta}$ where often .67 < β < .75 and 10 < k < 100 (empirical parameters)

i.e., the vocabulary size grows with more than the square root of the number of word tokens

Corpus	Tokens = N	Types = |V|	Avg
Switchboard phone conversations	2.4 million	20 thousand	120
Shakespeare	884,000	31 thousand	28.5
Corpus of Contemporary American English	440 million	2 million	220
Google N-grams	1 trillion	13+ million	76.9

Corpora

Words don't appear out of nowhere!

A text is produced by

- a specific writer (or specific writers),
- at a specific time,
- in a specific variety,
- of a specific language,
- for a specific function.

Corpora vary along dimensions like

- **Language**: 7097 languages in the world
- **Variety**, like African American Language (AAE) varieties.
  - AAE Twitter posts might include forms like "*iont" (I don't)*
- **Code switching**, e.g., Spanish/English, Hindi/English:
  - S/E: Por primera vez veo a @username actually being hateful! It was beautiful:)

  [For the first time I get to see @username actually being hateful! it was beautiful:)]

  - H/E: dost tha or ra- hega ...dont worry ...but dherya rakhe

  ["he was and will remain a friend ... don't worry ... but have faith"]
- **Genre:** newswire, fiction, scientific articles, Wikipedia
- **Author Demographics**: writer's age, gender, ethnicity, etc.

Corpus datasheets

- Whenever you build a corpus, you should be documenting these decisions in a datasheet for the corpus.

Motivation:

- Why was the corpus collected?
- By whom?
- Who funded it?

Situation: In what situation was the text written? (e.g. from work situations like doctors' notes, legal text, parliamentary or congressional proceedings, …)

Collection process: Data size? If it is a sub-sample how was it sampled? Was there consent? Pre-processing?

Annotation process, language variety, demographics, etc.

Text Normalization

- Every NLP task requires text normalization:
  1. Tokenizing (segmenting into) words
  2. Normalizing word formats
  3. Segmenting (into) sentences

Segmentation vs. Tokenization — Classical NLP Perspective

According to Jurafsky & Martin's textbook:

- Segmentation divides text into larger linguistic units (sentences, clauses, or words).
- Tokenization identifies minimal linguistic units (tokens) such as words or punctuation marks within segments.
- Morphological analysis (after tokenization) may further decompose tokens into morphemes.
- → Classical pipeline: Segmentation → Tokenization → Morphological Analysis.

Illustrative Examples

- **English Example**:
  - Sentence: “They’re studying linguistics.”
  - Sentence Segmentation → [They’re studying linguistics .]
  - Tokenization → ['They', 'are', 'studying', 'linguistics', '.']
  - Morphological decomposition → study + ing
- **Arabic Example**:
  - Sentence: يدرسون اللغة العربية
  - Sentence Segmentation → [يدرسون اللغة العربية] (whole sentence)
  - Tokenization → ['يدرسون', 'اللغة', 'العربية']
  - Morphological decomposition → درس+ ي+ ون(prefix, root, suffix)

Modern NLP Adaptation (Jurafsky & Martin)

Modern NLP systems often blur the classical distinction:

- Subword segmentation (e.g., BPE, WordPiece) integrates segmentation and tokenization for data-driven efficiency.
- Tokens may represent subwords (e.g., "happi##ness") rather than words or morphemes ("happy##ness").
- In Arabic, segmentation and tokenization interact with morphological richness (affixes, clitics e.g. و "and").

Space-based tokenization

- A very simple way to tokenize
  - For languages that use space characters between words
    - Arabic, Cyrillic, Greek, Latin, etc., based writing systems
  - Segment off a token between instances of spaces
  - Additionally, punctuation can be used for this.
- Unix tools for space-based tokenization
  - The "tr" (translate character) command

Simple Tokenization in UNIX

- Given a text file, output the word tokens and their frequencies

Change all non-alpha to a single newline

```
tr -sc 'A-Za-z' '\n' < shakes.txt
     | sort
     | uniq -c
```

Sort in alphabetical order

Merge and count each type

```
1945 A
  72 AARON
  19 ABBESS
   5 ABBOT
 ... ...
```

```
25 Aaron
 6 Abate
 1 Abates
 5 Abbess
 6 Abbey
 3 Abbot
... ...
```

-s stands for squeeze, meaning replace each sequence of a repeated character listed in the set with a single occurrence of that character

-c stands for complement, meaning "all characters *other than* letters"

The first step: tokenizing

```
tr -sc 'A-Za-z' '\n' < shakes.txt | head
    # head, displays the first few lines of
    # its input (10 by default).
THE
SONNETS
by
William
Shakespeare
From
fairest
creatures
We
...
```

The second step: sorting

```
tr -sc 'A-Za-z' '\n' < shakes.txt | sort | head

A
A
A
A
A
A
A
A
A
A
...
```

More counting (with Normalisation)

- Merging upper and lower case

```
tr 'A-Z' 'a-z' < shakes.txt | tr -sc 'A-Za-z'
'\n' | sort | uniq -c
```

- Sorting the counts

```
tr 'A-Z' 'a-z' < shakes.txt | tr -sc 'A-Za-z'
'\n' | sort | uniq -c | sort -n -r
```

```
  23243 the
  22225 i
  18618 and
  16339 to
  15687 of
  12780 a
  12163 you
  10839 my
  10005 in
   8954  d
```

Sorts lines numerically in reverse order

What happened here?

Issues in Tokenization

- Can't just blindly remove punctuation:
  - m.p.h., Ph.D., AT&T, …
  - prices ($45.55)
  - dates (01/02/06)
  - URLs (http://www.stanford.edu)
  - hashtags (#nlproc)
  - email addresses (someone@cs.colorado.edu)
- Clitic: a word that doesn't stand on its own
  - "are" in we're, French "je" in j'ai, "le" in "l'honneur"
- When should multiword expressions (MWE) be words?
  - Hassi Dallaah rock 'n' roll

Tokenization in NLTK

Bird, Loper and Klein (2009), *Natural Language Processing with Python*. O'Reilly

```
>>> text = 'That U.S.A. poster-print costs $12.40...'
>>> pattern = r'''(?x)    # set flag to allow verbose regexps
...     ([A-Z]\.)+        # abbreviations, e.g. U.S.A.
...   | \w+(-\w+)*        # words with optional internal hyphens
...   | \$?\d+(\.\d+)?%?  # currency and percentages, e.g. $12.40, 82%
...   | \.\.\.            # ellipsis
...   | [][.,;"'?():-_`]  # these are separate tokens; includes ], [
... '''
>>> nltk.regexp_tokenize(text, pattern)
['That', 'U.S.A.', 'poster-print', 'costs', '$12.40', '...']
```

Tokenization in languages without spaces

- Many languages (like Chinese, Japanese, Thai) don't use spaces to separate words!

- How do we decide where the token boundaries should be?

Word tokenization in Chinese

- Chinese words are composed of characters called "**hanzi**" (or sometimes just "**zi**")
- Each one represents a meaning unit called a **morpheme**.
- Each word has on average 2.4 of them.
- But deciding what counts as a word is complex and not agreed upon.

How to do word tokenization in Chinese?

- 姚明进入总决赛 "Yao Ming reaches the finals"
- 3 words?
  姚明 进入 总决赛
  YaoMing reaches finals
- 5 words?
  姚 明 进入 总 决赛
  Yao Ming reaches overall finals
- 7 characters? (don't use words at all):
  姚 明 进 入 总 决 赛
  Yao Ming enter enter overall decision game

Word tokenization / segmentation

- In Chinese it is common to just treat each character (zi) as a token.
  - So the **segmentation** step is very simple
- In other languages (like Thai and Japanese), more complex word segmentation is required.
  - The standard algorithms are neural sequence models trained by supervised machine learning.

Another option for text tokenization: Byte Pair Encoding

- Instead of
  - white-space segmentation
  - single-character segmentation
- **Use the data** to tell us how to tokenize.
- **Subword tokenization** (because tokens can be parts of words as well as whole words)

Subword tokenization

- Three common algorithms:
  - **Byte-Pair Encoding (BPE)** (Sennrich et al., 2016)
  - **Unigram language modeling tokenization** (Kudo, 2018)
  - **WordPiece** (Schuster and Nakajima, 2012)
- All have 2 parts:
  - A token **learner** that takes a raw training corpus and induces a vocabulary (a set of tokens).
  - A token **segmenter** that takes a raw test sentence and tokenizes it according to that vocabulary.

Byte Pair Encoding (BPE) token learner

Let a vocabulary be the set of all individual characters

= {A, B, C, D,…, a, b, c, d….}

- Repeat:
  - Choose the two symbols that are most frequently adjacent in the training corpus (say 'A', 'B')
  - Add a new merged symbol 'AB' to the vocabulary
  - Replace every adjacent 'A' 'B' in the corpus with 'AB'.
- Until *k* merges have been done.

Byte Pair Encoding (BPE) token learner

- BPE: algorithm often used in NLP, particularly in the context of neural network models like transformers.
- **Steps of the BPE Algorithm:**
  - **Initialize**: Start with a vocabulary of all characters in the training corpus.
  - **Count Pairs**: Count all pairs of adjacent tokens in text.
  - **Merge**: Find most frequent pair and merge it into a new token.
  - **Update**: Repeat the counting and merging process until reaching a predefined vocabulary size or no pairs remain.
- **Tokenization**: BPE creates subword units, whole words or parts of words. E.g., "lower" might be tokenized as "low" and "er" if these subwords are common in the training data.

BPE token learner algorithm

function BYTE-PAIR ENCODING(strings C, number of merges k) **returns** vocab V

$V \leftarrow$ all unique characters in C # initial set of tokens is characters
for $i = 1$ **to** k **do** # merge tokens til k times
  $t_L, t_R \leftarrow$ Most frequent pair of adjacent tokens in C
  $t_{NEW} \leftarrow t_L + t_R$ # make new token by concatenating
  $V \leftarrow V + t_{NEW}$ # update the vocabulary
  Replace each occurrence of t_L, t_R in C with t_{NEW} # and update the corpus
return V

Byte Pair Encoding (BPE) Addendum

Most subword algorithms are run inside space-separated tokens.

So we commonly first add a special end-of-word symbol '__' before space in training corpus

Next, separate into letters.

```
def add_end_of_word_symbol(text):
 words = text.split() # Split by spaces
 return ' '.join([word+'__' for word in words])


text = "Hello world! This is an example."
processed_text = add_end_of_word_symbol(text)
print(processed_text)
```

BPE token learner

Original (very fascinating 🙄) corpus:

low low low low low lowest lowest newer newer newer newer newer newer wider wider wider new new

Add end-of-word token, resulting in this vocabulary:

corpus	representation
5	l o w _
2	l o w e s t _
6	n e w e r _
3	w i d e r _
2	n e w _

vocabulary

_, d, e, i, l, n, o, r, s, t, w

BPE token learner

corpus

5 l o w _
2 l o w e s t _
6 n e w e r _
3 w i d e r _
2 n e w _

vocabulary

_, d, e, i, l, n, o, r, s, t, w

Merge e r to er

corpus

5 l o w _
2 l o w e s t _
6 n e w er _
3 w i d er _
2 n e w _

vocabulary

_, d, e, i, l, n, o, r, s, t, w, er

BPE

corpus

5 l o w _
2 l o w e s t _
6 n e w er _
3 w i d er _
2 n e w _

vocabulary

_, d, e, i, l, n, o, r, s, t, w, er

Merge er _ to er_

corpus

5 l o w _
2 l o w e s t _
6 n e w er_
3 w i d er_
2 n e w _

vocabulary

_, d, e, i, l, n, o, r, s, t, w, er, er_

BPE

corpus

5 l o w _
2 l o w e s t _
6 n e w er_
3 w i d er_
2 n e w _

vocabulary

_, d, e, i, l, n, o, r, s, t, w, er, er_

Merge n e to ne

corpus

5 l o w _
2 l o w e s t _
6 ne w er_
3 w i d er_
2 ne w _

vocabulary

_, d, e, i, l, n, o, r, s, t, w, er, er_, ne

BPE

The next merges are:

Merge	Current Vocabulary
(ne, w)	_, d, e, i, l, n, o, r, s, t, w, er, er_, ne, new
(l, o)	_, d, e, i, l, n, o, r, s, t, w, er, er_, ne, new, lo
(lo, w)	_, d, e, i, l, n, o, r, s, t, w, er, er_, ne, new, lo, low
(new, er_)	_, d, e, i, l, n, o, r, s, t, w, er, er_, ne, new, lo, low, newer_
(low, _)	_, d, e, i, l, n, o, r, s, t, w, er, er_, ne, new, lo, low, newer_, low_

BPE token segmenter algorithm

On the test data, run each merge learned from the training data:

- Greedily
- In the order we learned them
- (test data frequencies don't play a role, just the training data frequencies)

So: merge every e r to er, then merge er _ to er_, etc.

- Result:
  - Test set "n e w e r _" would be tokenized as a full word
  - Test set "l o w e r _" would be two tokens: "low er_"

Importance of BPE

- **Handling Rare Words**: BPE can effectively deal with out-of-vocabulary words by breaking them into sub-word units, allowing models to handle rare and unseen words more robustly.
- **Reducing Vocabulary Size**: By merging frequent pairs, BPE helps reduce the vocabulary size, which can improve the efficiency of models in terms of memory and computation.
- **Improved Performance**: BPE is used by many state-of-the-art models, like BERT and GPT, or similar algorithms, as they help capture morphological variations and improve generalization across languages.

Properties of BPE tokens

- These tokens usually include frequent words and frequent sub-words which are often morphemes like *-est* or *–er*

- A **morpheme** is the smallest meaning-bearing unit of a language.
  - *unlikeliest* has 3 morphemes *un-*, *likely*, and *-est*

Basic Text Processing: Word Normalization & other issues

- Putting words/tokens in a standard format (normalization)
  - U.S.A. or USA
  - uhhuh or uh-huh
  - Fed (i.e. US Federal Reserve Bank) or fed
  - am, is, be, are
- This is important, e.g. in Information Extraction and Information Retrieval.

Case folding

- Applications like IR: reduce all letters to lower case
  - Since users tend to use lower case
  - Possible exception: upper case in mid-sentence?
    - e.g., ***General Motors***
    - ***Fed*** vs. ***fed***
    - ***SAIL*** (as in Chevrolet Sail or the Canadian outdoor superstore) vs. ***sail***
- For sentiment analysis, MT, Information extraction
  - Case is helpful (***US*** versus ***us*** is important)

Lemmatization

Represent all words as their lemma, their shared root

= dictionary headword form:

- *am, are, is* → *be*
- *car, cars, car's, cars'* → *car*

- Spanish quiero ('I want'), quieres ('you want')

  → querer 'want'

- *He is reading detective stories*

  → *He be read detective story*

Lemmatization: done by Morphological Parsing

- Morphemes:
  - The small meaningful units that make up words
  - **Stems**: The core meaning-bearing units
  - **Affixes**: Parts that adhere to stems, often with grammatical functions
- Morphological Parsers:
  - Parse *cats* into two morphemes *cat* and *s*
  - Parse Spanish *amaren* (if in the future "they would love') into morpheme *amar* 'to love', and the morphological features *3PL* and *future subjunctive*.

Stemming

- Reduce terms to stems, chopping off affixes crudely

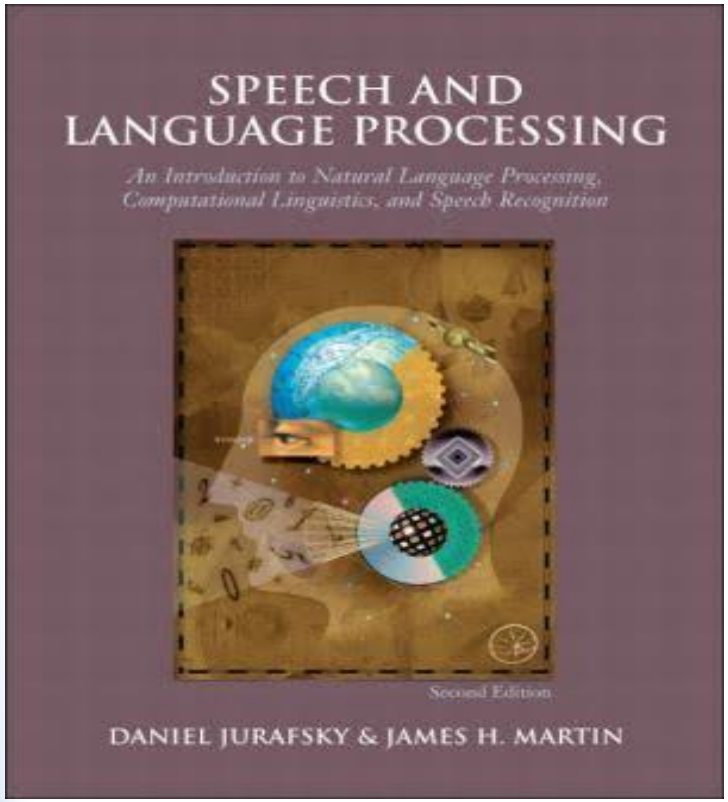
This was not the map we found in Billy Bones' chest, but an accurate copy, complete in all things-names and heights and soundings-with the single exception of the red crosses and the written notes.



Thi wa not the map we found in Billi Bone s chest but an accur copi complet in all thing name and height and sound with the singl except of the red cross and the written note .

Textbook

Jurafsky, D. and Martin, J.H. (2024) *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models* (3rd ed. draft), Prentice Hall.
https://web.stanford.edu/~jurafsky/slp3/ed3bookaug20_2024.pdf



The draft version of the 3rd edition will be used; it has been updated to include more recent topics such as Vector Semantics and Embeddings, ANNs and Deep Learning for NLP, Transformers, etc.

This course **slides** are largely based on this textbook and its authors' slides with light customisations whenever seen fit.

This chapter: